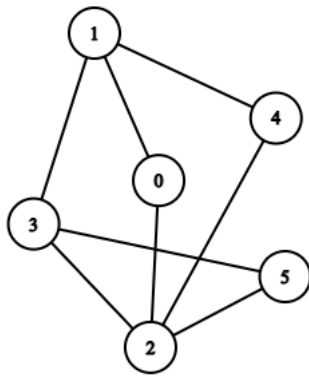




```

graph = {
  '0': ['1', '2'],
  '1': ['0', '3', '4'],
  '2': ['0', '3', '4', '5'],
  '3': ['1', '2', '5'],
  '4': ['1', '2'],
  '5': ['2', '3']
}

visited = []
queue = []

def bfs(start):
    visited.append(start)
    queue.append(start)

    while queue:
        node = queue.pop(0)
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.append(neighbour)
                queue.append(neighbour)

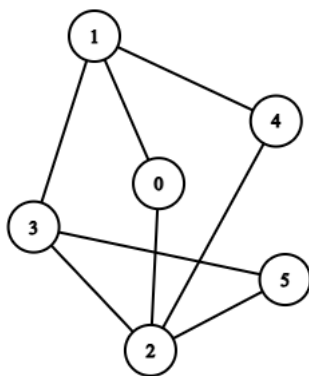
print("BFS Traversal:")
bfs('1')

```

```

BFS Traversal:
1 0 3 4 2 5

```



```

graph = {
  '0': ['1', '2'],
  '1': ['0', '3', '4'],
  '2': ['0', '3', '4', '5'],
  '3': ['1', '2', '5'],
  '4': ['1', '2'],
  '5': ['2', '3']
}

visited = []

def dfs(node):

```

```

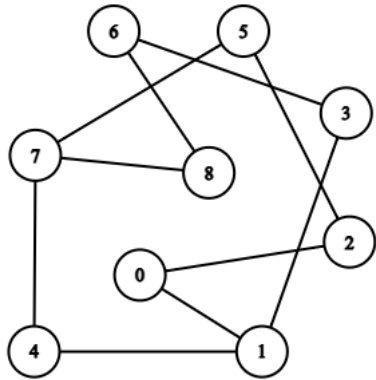
if node not in visited:
    print(node, end=" ")
    visited.append(node)

    for neighbour in graph[node]:
        dfs(neighbour)

print("DFS Traversal:")
dfs('1')

```

DFS Traversal:
1 0 2 3 5 4



```

graph = {
    '0': ['1', '2'],
    '1': ['0', '3', '4'],
    '2': ['0', '5'],
    '3': ['1', '6'],
    '4': ['1', '7'],
    '5': ['2', '7'],
    '6': ['3', '8'],
    '7': ['4', '5', '8'],
    '8': ['6', '7']
}

def bfs(graph, start):
    visited = set()
    queue = []

    visited.add(start)
    queue.append(start)

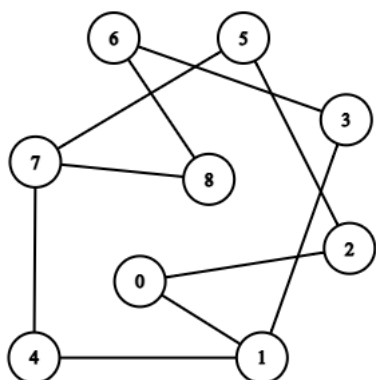
    while queue:
        node = queue.pop(0)
        print(node, end=" ")

        for neighbour in graph[node]:
            if neighbour not in visited:
                visited.add(neighbour)
                queue.append(neighbour)

print("BFS Traversal:")
bfs(graph, '0')

```

BFS Traversal:
0 1 2 3 4 5 6 7 8



```
graph={
    '0':['1', '2'],
    '1':['0', '3', '4'],
    '2':['0', '5'],
    '3':['1', '6'],
    '4':['1', '7'],
    '5':['2', '7'],
    '6':['3', '8'],
    '7':['4', '5', '8'],
    '8':['6', '7']
}

def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()

    print(node, end=" ")
    visited.add(node)

    for neighbour in graph[node]:
        if neighbour not in visited:
            dfs(graph, neighbour, visited)

print("\nDFS Traversal:")
dfs(graph, '0')
```

```
DFS Traversal:
0 1 3 6 8 7 4 5 2
```